Lab 2 | Transactional Application

You will find the starter files linked as “lab2.zip”.

While we provide a testing framework for most of your methods, the testing we provide is partial (although significant). It is up to you to implement your solutions so that they completely follow the provided specification.

Things to turn in:

- lab2_code.zip

0. Setup and General Specifications

We maintain the same general specification as lab 1. Refer to the previous document for details.

1. Database Design (10 points)

If your implementation for this lab works with your old database, submit your previous createTables.sql file.

If you make any changes to your database. Submit a new copy of your createTables.sql file such that we can recreate your database starting from a copy of the flights database.

2. Java Transactional Application (40 points)

Copy the relevant code from Lab 1 into this new lab.

For this lab, you must implement transactions for each of your methods. The general technique to implement this is to turn off auto-commit and, for each method in your application, ROLLBACK on any failure and COMMIT on success. For read-only methods, you can always COMMIT since nothing has changed in your database.

In the case of a database error, attempt to rollback. Depending on the situation, like a read-only method, you might also want to recursively retry the method.

The total number of lines of code that you will write for this lab is relatively small but getting everything to work harmoniously may take some time. Debugging transactions can be a pain, but print statements are your friend!

3. Transactional Test Cases (10 points)

We have provided the same test harness as before. Our test harness will compile your code and run the test cases in the folder you provide. To run the harness, execute in the lab2 folder:

```
mvn compile
mvn test
```

For every test case, it will either print pass or fail, and for all failed cases it will dump out what the implementation returned, and you can compare it with the expected output in the corresponding case file. Each test case file is of the below format. Note user1 and user2 are not necessarily different users in the application. The symbol “|” indicates a separator between potential sets of outputs for the last set of commands.

```
[user1 command 1]
[user1 command 2]
...
*
[user1 potential output1 line 1]
[user1 potential output1 line 2]
...
|
[user1 potential output2 line 1]
[user1 potential output2 line 2]
...
*
[user2 command 1]
[user2 command 2]
...
*
[user2 potential output1 line 1]
[user2 potential output1 line 2]
...
|
[user2 potential output2 line 1]
[user2 potential output2 line 2]
...
*
# everything following ‘#’ is a comment on the same line
```

Your task is to write at least 1 **parallel** test case (involving multiple application instances) for each of the 7 commands (you don't need to test quit). Separate each test case in its own file and name it <command name>_<some descriptive name for the test case>.txt and turn them in along with the original test cases. It’s fine to turn in test cases for erroneous conditions (e.g., booking on a full flight, paying the same reservation from two sessions, etc).